



Searching and Sorting Algorithms Review

Searching Algorithms

Searching algorithms let you search for a target within a certain list of items.

In computer science, there are two searching algorithms you should know about. They are **linear search** and **binary search** algorithms. It is important to understand the differences between the two algorithms and when to use each one.

Linear Search

- Linear search iterates through a list of items, checking each item from the beginning until the target item is found.
- If the target item is found, the index where it is located in the list is returned.
- If the target is not found, it returns `-1`, which means **invalid index** in most programming languages.
- Because linear search checks each item until it finds the target, it is not efficient for a large list of items.
- The time complexity of linear search is $O(n)$ because the time it takes to search through the list grows linearly with the size of the list.
- The space complexity of linear search is $O(1)$ because it doesn't require any additional space to search through the list.

Binary Search

- Binary search works by dividing a list of items in half, and checking if the target value is in the middle of the list.
- The condition for binary search to work is that the items in the list must be sorted, either in ascending or descending order.
- Binary search is a more efficient algorithm for searching through a large list of items because it divides the list of items in half and ignores any half where the target is not found.
- If the target item is found in the middle of the list, the index of the target item is returned.
- If the item is not found, the algorithm checks if the target item is in the left or right half of the list.
- It continues to divide the remaining parts of the list into halves until the target item is found.
- If the target item is finally not found in the list, it returns `-1`.
- The time complexity of binary search is $O(\log n)$ because the time it takes to search through the list grows logarithmically with the size of the list.
- The space complexity of binary search is $O(1)$ because it doesn't require any additional space to search through the list.

How Linear Search Differs from Binary Search

- Binary search is more suitable for a large list of items compared to linear search.
- The time complexity of linear search is $O(n)$ because the time it takes to search through the list grows linearly with the size of the list.
- The time complexity of binary search is $O(\log n)$ because the time it takes to search through the list grows logarithmically with the size of the list.

Sorting Algorithms and Divide-and-Conquer

In computer science, divide-and-conquer is a technique used to break down a problem into smaller sub-problems so they are easier to solve. Recursion is the technique often employed in divide-and-conquer, and divide-and-conquer is a powerful strategy used to implement many efficient sorting algorithms like merge sort.

Merge Sort

- Merge sort is a sorting algorithm that follows the divide-and-conquer approach.
- It works by recursively dividing a list into smaller sub-lists until each sub-list contains only one element.
- It then repeatedly merges the sub-lists back together in a sorted order.
- The time complexity for merge sort is $O(n \log n)$ because the list is continuously divided in half $(\log n)$ and then merged together $(O(n))$.
- The space complexity of merge sort is $O(n)$ because it is not an in-place sorting algorithm.